

Actividad 7.

Didier Eduardo Coa

Lenguaje Programacion II

Universidad de la salle

19/5/2026

Manual de Programación

Autor: Eduardo Coa

Repositorio: Eduardo-Coa/actividad-7-phq9

Rama principal de trabajo: main

1. Descripción General

Este proyecto implementa un sistema **CRUD** completo sobre la entidad **CuestionarioPHQ9**, cuestionario clínico de detección de síntomas depresivos (*Patient Health Questionnaire – 9*) aplicado a estudiantes universitarios. El sistema está desarrollado en **Python 3.10+** aplicando principios SOLID, Clean Code y PEP 8, con persistencia en formato **JSON** e interfaz de usuario en consola (CLI) basada en las 10 heurísticas de Nielsen.

Sistema / Módulo	Descripción
Modelo (Model)	Clase CuestionarioPHQ9: encapsula los datos del cuestionario, valida entradas y calcula automáticamente puntaje total y nivel de severidad en <code>__post_init__</code> .
Repositorio DAO	Interfaz abstracta <code>IRepositorioCuestionario</code> (ABC) con 6 métodos CRUD e implementación concreta <code>CuestionarioJsonRepositorio</code> que persiste en <code>data/cuestionarios.json</code> .
Servicio (Service)	Clase <code>CuestionarioServicio</code> : orquesta la lógica de negocio, genera UUID para cada registro, ordena resultados y calcula estadísticas.
Controlador (Controller)	Clase <code>CuestionarioControlador</code> : mediador MVC entre la Vista y el Servicio; convierte excepciones de dominio en tuplas (<code>bool</code> , <code>str</code> , <code>Optional[T]</code>).
Vista (View)	Clase <code>MenuPrincipal</code> : interfaz CLI interactiva con menú numerado, validaciones inmediatas y las 10 heurísticas de Nielsen.
Excepciones de dominio	Jerarquía <code>AppError</code> → <code>FechaInvalidaError</code> , <code>PuntajeInvalidoError</code> , <code>CuestionarioNoEncontradoError</code> , <code>EstudianteDuplicadoError</code> .

La arquitectura combina el patrón **MVC** para separar presentación, lógica y datos, con el patrón **DAO/Repository** para abstraer el acceso a datos detrás de una interfaz, permitiendo cambiar el mecanismo de persistencia sin modificar ninguna capa superior.

2. Estructura del Proyecto

```

Actividad 7/
├── main.py                                ← Composition root (punto de entrada)
├── app/
│   ├── exceptions.py                     ← Jerarquía de excepciones de dominio
│   ├── utils/
│   │   └── constantes_negocio.py         ← Constantes PHQ-9 (umbrales, formatos)
│   ├── models/
│   │   └── cuestionario_phq9.py          ← Entidades CuestionarioPHQ9 y Respuesta
│   ├── interfaces/
│   │   └── i_repositorio_cuestionario.py ← Contrato ABC con 6 métodos CRUD
│   ├── repositories/
│   │   └── cuestionario_json_repositorio.py ← Implementación DAO → JSON
│   ├── services/
│   │   └── cuestionario_servicio.py      ← Lógica de negocio (CRUD + estadísticas)
│   ├── controllers/
│   │   └── cuestionario_controlador.py   ← Mediador MVC
│   └── ui/
│       ├── mensajes.py                  ← Constantes de texto visible al usuario
│       └── menu_principal.py             ← CLI interactivo (heurísticas Nielsen)
├── data/
│   └── cuestionarios.json                ← Almacenamiento persistente JSON
└── tests/
    ├── conftest.py                      ← Fixtures compartidos
    ├── test_model.py                    ← 12 casos: modelo y serialización
    ├── test_repositorio.py              ← 4 casos: persistencia JSON
    └── test_servicio.py                  ← 4 casos: lógica de negocio

```

3. Patrones de Diseño: MVC y DAO/Repository

3.1 Patrón MVC (Model – View – Controller)

Definición formal: MVC es un patrón arquitectónico que divide una aplicación en tres componentes con responsabilidades bien delimitadas. El *Modelo* gestiona los datos y la lógica de dominio sin conocer la presentación; la *Vista* presenta la información y captura la entrada del usuario sin contener lógica de negocio; el *Controlador* recibe las acciones del usuario desde la Vista, las delega al Modelo o al Servicio, y devuelve el resultado a la Vista. El objetivo es lograr bajo acoplamiento y alta cohesión entre las capas.

(Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", 1994.)

Rol MVC	Clase en el proyecto	Archivo	Responsabilidad
Model	CuestionarioPHQ9	app/models/cuestionario_phq9.py	Entidad de dominio: valida datos, calcula puntaje y clasifica severidad. No conoce la vista ni el repositorio.
View	MenuPrincipal	app/ui/menu_principal.py	Imprime menús, lee input, muestra resultados. No contiene lógica de negocio ni accede a datos.
Controller	CuestionarioControlador	app/controllers/cuestionario_controlador.py	Parsea la entrada de la Vista, llama al Servicio y convierte errores en mensajes legibles. No imprime ni persiste.

3.2 Patrón DAO / Repository

Definición formal: El patrón *Data Access Object* (DAO) define una interfaz que abstrae el acceso a una fuente de datos. Toda la lógica de persistencia se encapsula en un objeto DAO concreto, de modo que el resto del sistema depende únicamente del contrato abstracto y no del mecanismo de almacenamiento específico (archivo, base de datos, memoria). En la variante *Repository* (DDD, Domain-Driven Design), el repositorio actúa como una colección en memoria de objetos de dominio, ocultando completamente los detalles de persistencia. (Fowler, "Patterns of Enterprise Application Architecture", 2002.)

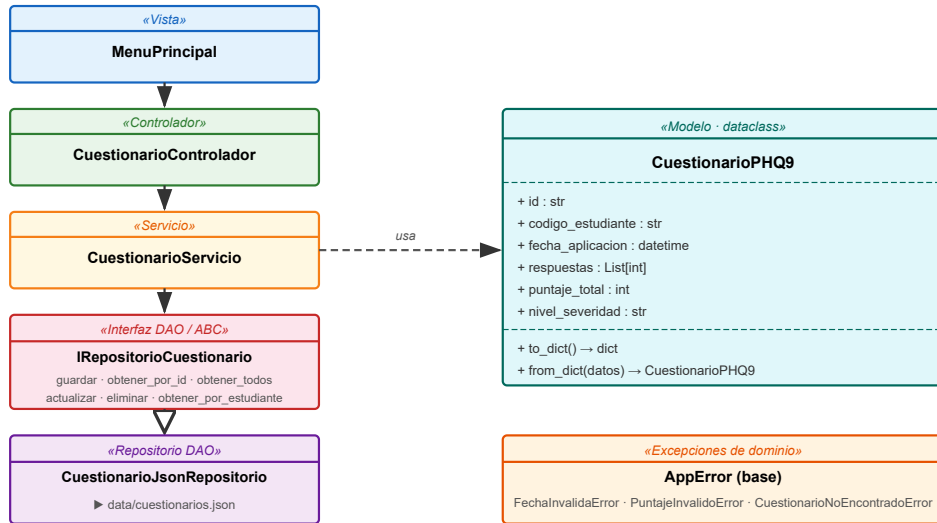
Componente DAO	Clase	Archivo	Rol
Contrato abstracto	IRepositorioCuestionario	app/interfaces/i_repositorio_cuestionario.py	ABC con 6 métodos abstractos: guardar, obtener_por_id, obtener_todos, actualizar, eliminar, obtener_por_estudiante.
Implementación concreta	CuestionarioJsonRepositorio	app/repositories/cuestionario_json_repositorio.py	Implementa los 6 métodos usando json.load / json.dump sobre data/cuestionarios.json.
Consumidor (Servicio)	CuestionarioServicio	app/services/cuestionario_servicio.py	Recibe IRepositorioCuestionario por inyección de dependencias. Nunca instancia el repositorio directamente.

4. Principios SOLID

Principio	Definición	Evidencia en el proyecto
S — Single Responsibility	Cada clase debe tener una única razón para cambiar.	CuestionarioPHQ9 solo modela datos; CuestionarioJsonRepositorio solo persiste; MenuPrincipal solo presenta. Ninguna clase mezcla responsabilidades.
O — Open / Closed	Abierto para extensión, cerrado para modificación.	Para agregar persistencia en BD basta crear CuestionarioSQLiteRepositorio(IRepositorioCuestionario) sin modificar ningún archivo existente.
L — Liskov Substitution	Las subclases deben poder sustituir a su base sin romper el comportamiento.	Los tests de servicio usan MagicMock() como sustituto total del repositorio real y el servicio funciona igual. Ver tests/test_servicio.py.
I — Interface Segregation	Interfaces pequeñas y específicas; no forzar dependencias innecesarias.	IRepositorioCuestionario define exactamente los 6 métodos del dominio PHQ-9. No existe un repositorio genérico con métodos de otras entidades.
D — Dependency Inversion	Depender de abstracciones, no de implementaciones concretas.	CuestionarioServicio.__init__(self, repositorio: IRepositorioCuestionario). El único lugar con instancias concretas es main.py (composition root).

5. Sistema PHQ-9 — CRUD

5.1 Diagrama de Clases



5.2 Descripción de Clases

Clase	Responsabilidad
CuestionarioPHQ9	Dataclass que modela el cuestionario PHQ-9. Valida ID, código de estudiante, fecha (no futura) y 9 respuestas (0-3). Calcula puntaje_total y nivel_severidad en __post_init__. Serializa con to_dict() / from_dict().
Respuesta	Dataclass auxiliar para un ítem individual. Valida numero_pregunta ∈ [1,9] y valor_respuesta ∈ [0,3].
IRepositorioCuestionario	Clase base abstracta (ABC) que define el contrato de 6 operaciones CRUD. Ningún consumidor depende de implementaciones concretas.
CuestionarioJsonRepositorio	Implementación DAO. Lee y escribe el JSON con json.load / json.dump. Crea el archivo y directorio automáticamente. Acepta la ruta como parámetro para facilitar tests con tmp_path.
CuestionarioServicio	Lógica de negocio. Genera UUID por registro, ordena por fecha descendente y calcula estadísticas (distribución de severidad, promedio, casos de riesgo).
CuestionarioControlador	Mediador MVC. Parsea la fecha del formato de UI, llama al servicio y convierte todas las excepciones de dominio en tuplas (bool, str, Optional[T]). La vista no hace try/except.
MenuPrincipal	Vista CLI con 7 acciones de menú. Valida formatos en el momento del ingreso, muestra progreso "Ítem N/9", ofrece cancelación en todo campo y confirma antes de eliminar. Implementa las 10 heurísticas de Nielsen.
AppError y subclases	Jerarquía de excepciones: FechaInvalidaError, PuntajeInvalidaError, CuestionarioNoEncontradoError, EstudianteDuplicadoError. Todas heredan de AppError.

5.3 Interfaz ABC — IRepositorioCuestionario

Método	Firma completa	Descripción
guardar	(cuestionario: CuestionarioPHQ9) → None	Persiste un nuevo cuestionario. Lanza EstudianteDuplicadoError si ya existe registro del mismo estudiante en la misma fecha calendario.
obtener_por_id	(id_cuestionario: str) → CuestionarioPHQ9	Recupera por ID único. Lanza CuestionarioNoEncontradoError si no existe.
obtener_todos	() → List[CuestionarioPHQ9]	Retorna todos los cuestionarios almacenados (lista puede estar vacía).
actualizar	(cuestionario: CuestionarioPHQ9) → None	Reemplaza el registro con el mismo ID. Lanza CuestionarioNoEncontradoError si no existe.
eliminar	(id_cuestionario: str) → None	Elimina por ID. Lanza CuestionarioNoEncontradoError si no existe.
obtener_por_estudiante	(codigo_estudiante: str) → List[CuestionarioPHQ9]	Filtra por código de estudiante (case-insensitive). Lista vacía si no hay registros.

5.4 Métodos adicionales por clase

Clase	Método	Descripción
CuestionarioPHQ9	es_riesgo_severo: bool (property)	Retorna True si puntaje_total ≥ 20 .
CuestionarioPHQ9	to_dict() → dict	Serializa a diccionario JSON-compatible. Fecha como ISO 8601.
CuestionarioPHQ9	from_dict(datos: dict) → CuestionarioPHQ9	Classmétodo. Deserializa desde diccionario; convierte fecha con datetime.fromisoformat().
CuestionarioServicio	obtener_estadisticas() → dict	Retorna total de registros, promedio de puntaje, distribución por nivel de severidad y casos severos.
CuestionarioServicio	buscar_por_estudiante(codigo) → List	Delega al repositorio y ordena resultados por fecha descendente.
MenuPrincipal	_leer_fecha() → Optional[str]	Lee y valida inmediatamente el formato DD/MM/AAAA HH:MM. Reintenta en bucle si es incorrecto.
MenuPrincipal	_leer_entero_en_rango(prompt, min, max) → Optional[int]	Lee un entero en el rango especificado con reintentos automáticos ante entradas inválidas.

5.5 Lógica de Negocio — Clasificación de Severidad PHQ-9

El puntaje total es la **suma de las 9 respuestas** (valor 0–3 por ítem; máximo posible: 27). La clasificación sigue los umbrales clínicos estándar del PHQ-9:

Nivel de Severidad	Rango de Puntaje	Constante en código
Mínimo	0 – 4	<i>puntaje < PUNTAJE_RIESGO_LEVE_PHQ9</i>
Leve	5 – 9	PUNTAJE_RIESGO_LEVE_PHQ9 = 5
Moderado	10 – 14	PUNTAJE_RIESGO_MODERADO_PHQ9 = 10
Moderadamente severo	15 – 19	PUNTAJE_RIESGO_MODSEVERO_PHQ9 = 15
Severo	20 – 27	PUNTAJE_RIESGO_SEVERO_PHQ9 = 20

5.6 Visualización de Pruebas Unitarias

test_model.py — Modelo y Serialización (12 casos)

<i>TestCuestionarioPHQ9Puntaje</i>	
test_puntaje_calculado_automaticamente	PASSED
test_clasificacion_severidad[Minimo]	PASSED
test_clasificacion_severidad[Leve]	PASSED
test_clasificacion_severidad[Moderado]	PASSED
test_clasificacion_severidad[Moderadamente severo]	PASSED
test_clasificacion_severidad[Severo]	PASSED
<i>TestCuestionarioPHQ9Validacion</i>	
test_fecha_futura_lanza_excepcion	PASSED
test_respuestas_fuera_de_rango_lanza_excepcion	PASSED
test_id_vacio_lanza_excepcion	PASSED
<i>TestCuestionarioPHQ9Serializacion</i>	
test_serializacion_round_trip	PASSED
<i>TestRespuesta</i>	
test_respuesta_valor_invalido_lanza_excepcion	PASSED
test_respuesta_numero_fuera_de_rango	PASSED

test_repositorio.py — Persistencia JSON (4 casos)

<i>TestRepositorioGuardar</i>	
test_guardar_y_obtener_por_id	PASSED
test_guardar_duplicado_mismo_dia_lanza_excepcion	PASSED
<i>TestRepositorioEliminar</i>	
test_eliminar_lanza_error_si_no_existe	PASSED
<i>TestRepositorioActualizar</i>	
test_actualizar_persiste_nuevas_respuestas	PASSED

test_servicio.py — Lógica de Negocio (4 casos)

<i>TestServicioRegistrar</i>	
test_registrar_asigna_uuid_unico	PASSED
<i>TestServicioListar</i>	
test_listar_ordena_por_fecha_descendente	PASSED
<i>TestServicioActualizar</i>	
test_actualizar_respuestas_recalcula_severidad	PASSED
<i>TestServicioEstadisticas</i>	
test_estadisticas_distribucion_correcta	PASSED

Resultado Final — Suite Completa

```

===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 rootdir:
C:\Users\USUARIO\Desktop\Actividad 7 collected 20 items tests/test_model.py
..... [ 60%] tests/test_repositorio.py .... [ 80%] tests/test_servicio.py
.... [100%] ===== 20 passed in 0.05s
=====

```


Archivo de Persistencia — data/cuestionarios.json

```
[
  {
    "id": "2baabc25-8a6d-4206-b02b-b0194fe8d95e",
    "codigo_estudiante": "EST-2026-002",
    "fecha_aplicacion": "2026-05-19T16:30:00",
    "respuestas": [3, 2, 3, 2, 3, 2, 3, 2, 3],
    "puntaje_total": 23,
    "nivel_severidad": "Severo"
  },
  {
    "id": "120e050a-1a0c-4665-9b75-33ab4cd9345a",
    "codigo_estudiante": "EST-2026-003",
    "fecha_aplicacion": "2026-05-19T16:30:00",
    "respuestas": [1, 1, 0, 1, 0, 1, 0, 1, 0],
    "puntaje_total": 5,
    "nivel_severidad": "Leve"
  },
  {
    "id": "91b600cd-1b33-4146-9b35-d24ce648ebb8",
    "codigo_estudiante": "EST-2026-001",
    "fecha_aplicacion": "2026-05-19T13:50:00",
    "respuestas": [0, 1, 0, 2, 0, 1, 2, 1, 2],
    "puntaje_total": 9,
    "nivel_severidad": "Leve"
  },
  {
    "id": "327d0fbf-21fb-4b1a-96d9-4b0f9279fd56",
    "codigo_estudiante": "EST-2026-004",
    "fecha_aplicacion": "2026-05-19T14:00:00",
    "respuestas": [3, 2, 3, 2, 3, 2, 0, 3, 2],
    "puntaje_total": 20,
    "nivel_severidad": "Severo"
  },
  {
    "id": "730c53b4-9688-4919-8744-a56e1b976066",
    "codigo_estudiante": "EST-2026-005",
    "fecha_aplicacion": "2026-05-19T15:00:00",
    "respuestas": [0, 1, 0, 1, 0, 1, 0, 1, 0],
    "puntaje_total": 20,
    "nivel_severidad": "Severo"
  }
]
```